PYTHON

- ending extension of file -> `.py`
- for single line comments use -> hashtag `#`
- for multiline comments use -> `''' xxx '''`

VARIABLES

case specific containers to store the data or values throughout the scope

DATA TYPES

1. integers `(int)` -> numbers
2. floats `(float)` -> decimal / fractional values
3. string `(str)` -> words enclosed in either `'xx'` OR `"xx",` text
4. boolean `(bool)` -> `True` or `False`

- `type(variable_name)` -> shows the data type of the variable
- everything in python is an object -> OOP paradigm so each type has different methods (functions) for same operator or same named function
- eg: `+` operator adds two ints/floats/bools
- while `+` concatanates two strings

PYTHON LISTS

- special data type (list)
- lists are basically arrays of C, C++
- **BUT** lists can store different data types as opposed to only single in C and C++
- also 0 indexed
- use `list[-1]` to access the last element of the list

LIST SLICING

- accessing only certain elements of the list
- SYNTAX: `list_name[starting index(inclusive) : ending index(exclusive)]`
- `[:]` to select all elements
- `[start:]` to select all elements from starting index
- `[:end]` to select all elements before ending index

LIST MANIPULATION

- to change single: `list_name[index] = new_value`
- to change multiple: `list_name[start:end] = [new val1, new val2, ...]`
- to add element/elements: `list_name + [values]`
- to remove elements: del `list_name[index or start and end]`
- `list2 = list1` -> here list2 is a reference for list1
- i.e. they both point to the same list (list2 is not a separate entity)
- to actually copy the data of list1 into a new list2 then: - `list2 = list1[:]` OR `list2 = list(list1)`

FUNCTIONS

- a piece of reusable code to perform a specific task eg: `type()` , `help()` etc

SOME INBUILT FUNCTIONS

- `print(content)` -> displays the content

- `help(function)` -> displays the documentation of the function written inside it
- `max(values or list)` -> selects the maximum value
- `round(num, places)` -> rounds up the number to the specified decimal digits, default = 0

HOW TO FIND THE FUNCTIONS?

- for standard tasks there most likely will be a function
- search it in the internet or ask gpt or read documentation

METHODS

- specific tasks or functions of specific objects
- accessed by the dot operator `object_name.method_name()`

LIST METHODS

- `max()` -> to find the maximum value of a list
- `index(value)` -> to get the index of value from the list
- `count(value)` -> counts the number of times value appears in the list
- `append(value)` -> adds value to the list at the end

STRING METHODS

- `capitalize()` -> capitalizes the first letter of the string
- `replace("old_val","new_val")` -> replaces the old_val from the string by a new_val
- `index("val")` -> displays the index of val from the string as string is also a character list

PACKAGES

- collection of different functions and methods packaged as a group
- NEED: if all the functions and methods are already prebuilt and installed, then the codebase would be very messy and there would be a lot of code that you wouldnt use that is why packages are necessary
- Directory of Python Scripts
- Each script = module
- Specify functions, methods,types
- Thousands of packages available: NumPy, Matplotlib, scikit-learn, etc

TO INSTALL A PACKAGE

- Download `get-pip.py`
- IN Terminal:
- `python3 get-pip.pypip3`
- `install numpy`

NUMPY

- numpy : NUMeric PYthon
- `numpy` is a python package which consists of `numpy.array` (a datatype similar to a python list but has much more advanced functions)
- used in data science for mathematical operations over the entire collection and
- faster speed

IMPORTING NUMPY AND ARRAYS

- `import numpy` -> imports all the contents of numpy package in the python program
- `import numpy as np` -> same as above but instead of numpy, we use np to access it *BEST PRACTICE*
- `from numpy import array` -> only imports array and its methods into the program

NP.ARRAY VS LIST

- list can contain multiple data types in one single list
- whereas np.array can only contain same datatype i.e. is homogeneous

eg:

```
import numpy as np

my_list = [1, "is", True]       # contains different data types
my_array = np.array(my_list)    # converts the list to array

print(my_array)                 # here all the elements of list are converted to array and the data type
is str
```

```
['1' 'is' 'True']
```

- they are different data types so they have different functions for same operators eg:

```
import numpy as np

list1 = [1,2,3]
newlist = list1 + list1
print(newlist)                  # + operator concatanates two lists

arr1 = np.array(list1)
newarr = arr1 + arr1
print(newarr)                   # + operator adds respective elements of the arrays
```

```
[1, 2, 3, 1, 2, 3]
[2 4 6]
```

2D ARRAYS and Subsetting

- 2D numpy arrays has rows and columns: eg: `matrix = np.array([[1,0,0], [0,1,0], [0,0,1]])`
- number of rows and columns can be checked by the shape method `matrix.shape`

accessing np rows, columns and elements

- `matrix[0]` OR `matrix[0,:]` -> accesses the first row
- `matrix[:,2]` -> accesses the third column
- `matrix[1,1]` OR `matrix[1][1]` -> accesses the second element of the second row
- `matrix[0:2, -1]` -> last element of first and second row

STATISTICS IN PYTHON

- `np.mean(numpy_array[range])` -> mean of the specified array elements
- `np.median(numpy_array[range])` -> median of the specified array elements
- `np.std(numpy_array[range])` -> standard deviation
- `np.corrcoef(numpy_array1[range], numpy_array2[range])` -> correleation coefficient between two array elements
- others include: `np.sum()`, `np.sort()`

GENERATE RANDOM DATA

- `np.random.normal(mn,sd,n)`
- where:
- mn = mean of the distribution of data to be generated
- sd = standard deviation of the distribution of data
- n = number of samples

eg: `values = np.random.normal(1.75, 0.20, 5000)`, `weight = np.round(np.random.normal(60.32, 15, 5000), 2)`

Stacking two lists -> `new_list = np.column_stack((list1, list2))`

MATPLOTLIB

- used in data visualization, data analysis: to explore data and report insights

IMPORTING MATPLOTLIB

- `import matplotlib.pyplot as plt`

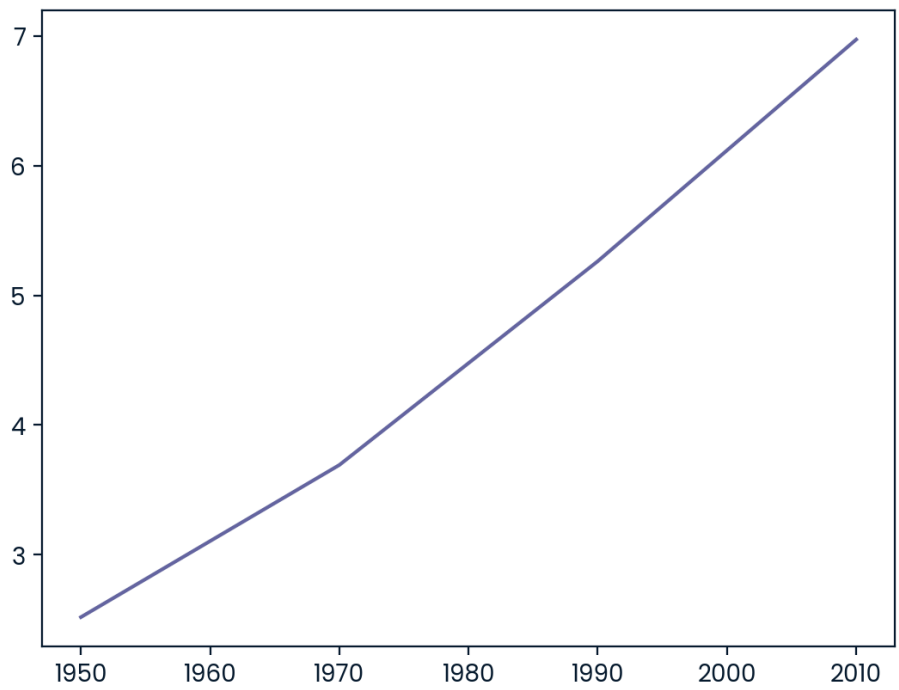
LINE PLOT

- basically a line graph
- use `plt.plot()` to make the line graph

```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

# use of .plot() function to make the graph
plt.plot(year, pop)

# shows the graph only when .show() is called
plt.show()
```



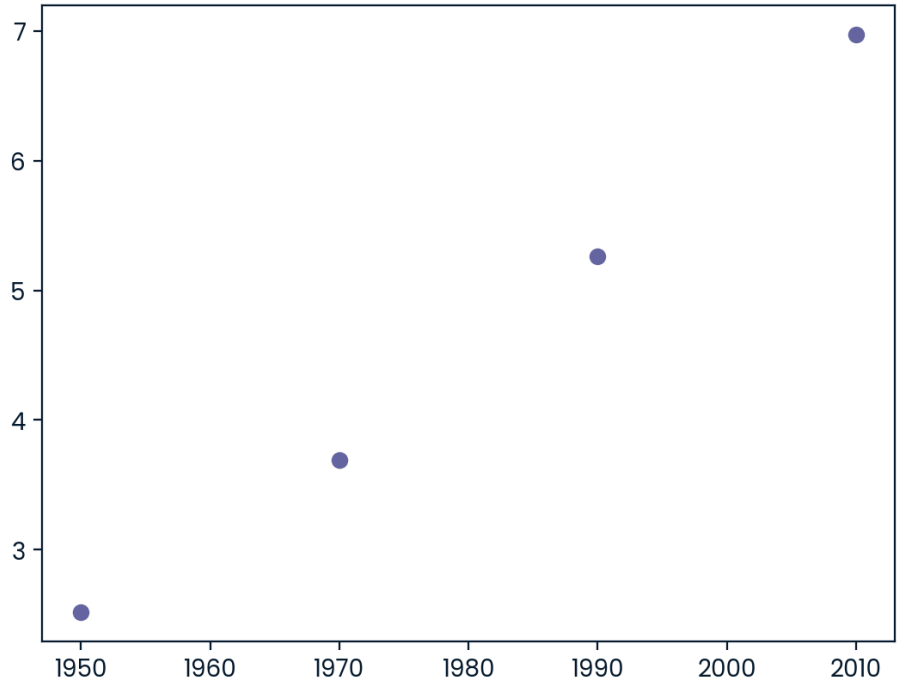
SCATTER PLOT

- a scatter plot
- use `plt.scatter()` to make the scatter plot

```
import matplotlib.pyplot as plt
year = [1950, 1970, 1990, 2010]
pop = [2.519, 3.692, 5.263, 6.972]

# use of .scatter() to create a scatter plot
plt.scatter(year, pop)

# .show() to display the plot
plt.show()
```



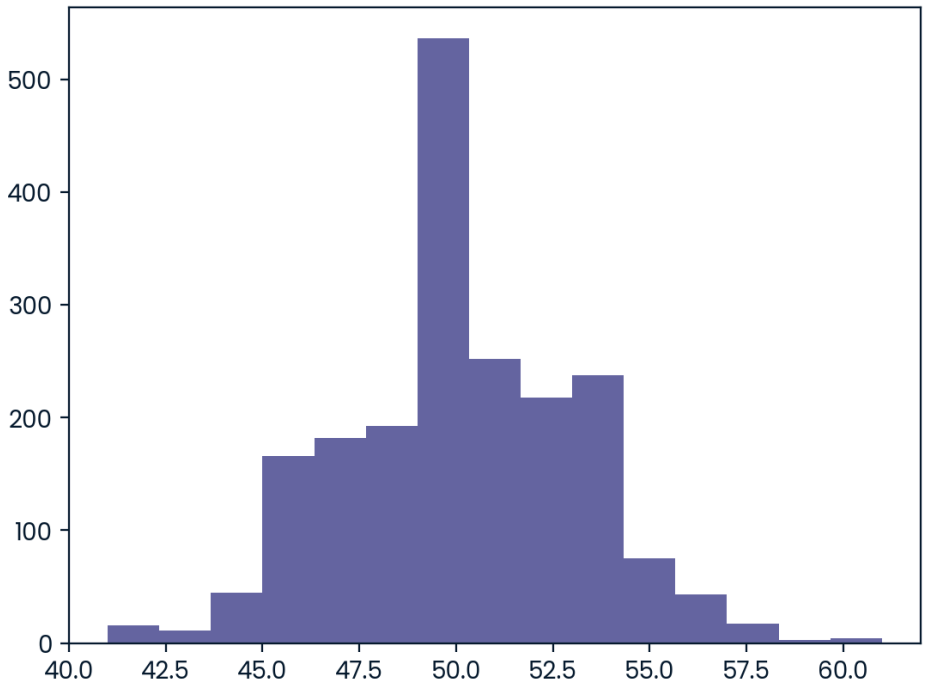
HISTOGRAM

- a way to visualize dataset and distribution by compartmentalisation in the range
- `plt.hist()` to create a histogram
- can specify the values you want to add and the number of compartments to make (called bins)
- eg: `plt.hist(values, bins = 5)` -> creates a histogram of values in 5 equal compartments

```
import matplotlib.pyplot as plt
import numpy as np

# generates 2000 random values with average 50 and of sd = 3
values = np.round(np.random.normal(50, 3, 2000))

# creates a histogram by dividing the data into 15 bins
plt.hist(values, bins = 15)
plt.show()
```



CUSTOMISATION OF DATA

- *NOTE:* MUST BE USED IN BETWEEN `plt.plot()` AND `plt.show()`
- `plt.xlabel('X-axis')`, `plt.ylabel('Y-axis')` -> naming the axes
- `plt.title('Graph')` -> title of the graph
- `plt.xticks([values of each units in x-axis])`, `plt.yticks([units in y axis])` -> to specify the increments in values eg: 1 unit = 2mA in physics

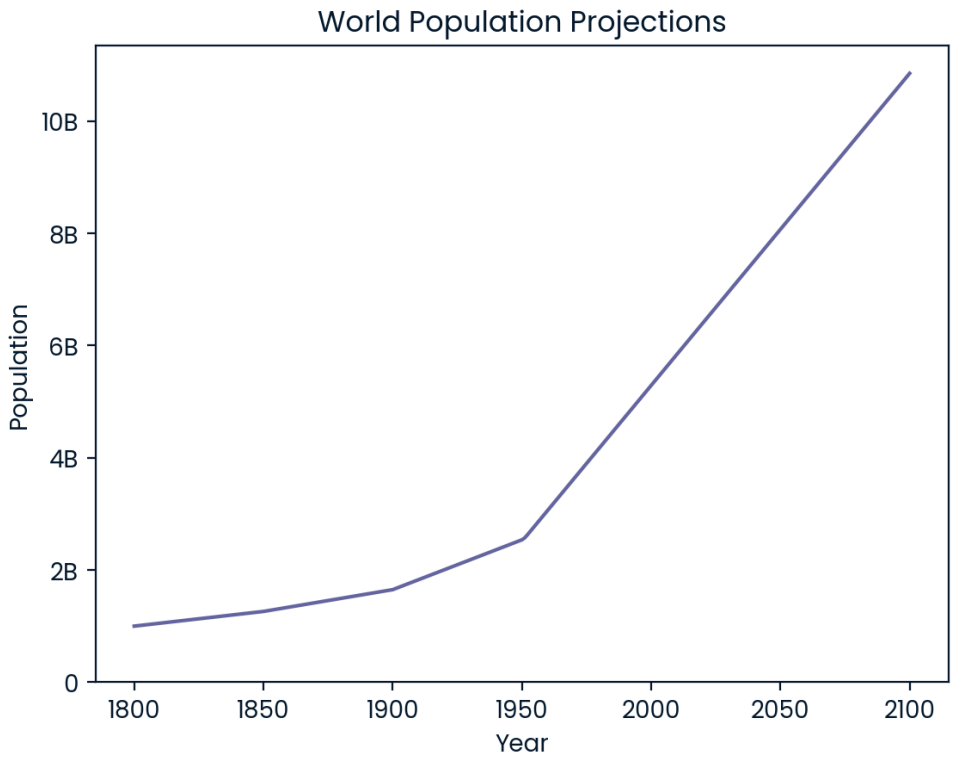
```
import matplotlib.pyplot as plt

year = [1950, 1951, 1952, 2100]
pop = [2.538, 2.57, 2.62, 10.85]
# Add more data
year = [1800, 1850, 1900] + year
pop = [1.0, 1.262, 1.650] + pop

plt.plot(year, pop)

# Customisation
plt.xlabel('Year')
plt.ylabel('Population')
plt.title('World Population Projections')

# the first list is the increment list
# while the second is the one that will be displayed in the graph
plt.yticks([0, 2, 4, 6, 8, 10],           ['0', '2B', '4B', '6B', '8B', '10B'])
plt.show()
```



DICTIONARY

- another data type in python
- used to store data in a key-value pair manner
- kind of like a lookup table
- select, remove and update with `[]`
- indexed by keys rather than numbers
- *NOTE:* Keys have to be "immutable" objects i.e. ints, floats, strings Cant be lists

ADDING AND REMOVING ITEMS IN DICTIONARY

- `dict = {key1:value1, key2:value2}` -> declaring a dictionary

- `dict[key] = value` -> adding a new item to the dictionary
- `del(dict[key])` -> removes the existing key value pair from the dictionary
- `dict[key]` -> returns the value of the key value pair
- `key in dict` -> checks if the given key is in the dictionary or not and returns the bool result

PANDAS

- it is another library in python that works with datasets (tables) and dataseries (rows of tables, values)
- high level data manipulation tool

TO READ DATA IN PANDAS

- `import pandas as pd` -> imports the pandas library
- `df_name = pd.DataFrame(dict_name)` -> converts a python dict to a pandas df
- `df_name = pd.read_csv("path/to/csv_file.csv", index_col = 0)` -> reads a CSV file as a pandas df and indexes the column 0 as reference column

Indexing and selecting Data in Pandas

- `df_name["column_name"]` -> access the column as a dataseries
- `df_name[["column_name"]]` -> access the column and the index as a dataframe
- `df_name[["col_1", "col_2"]]` -> dataframe containing two columns
- `df_name[start:end]` -> accessing rows from start to end-1

ROW ACCESS: LOC and ILOC

- loc: label based row access
- iloc: integer position based row access

ROW ACCESS LOC

- NOTE: index: specific title for a row, could be integer like: 0,1,2 or sth specific like: 'BR', 'NP', 'IN' for countries
- `df_name.loc[row_index]` -> access row as dataseries
- `df_name.loc[[row_index]]` -> access row as dataframe
- `df_name.loc[[r_index1, r_index2, r_index3]]` -> access multiple rows as dataframe
- `df_name.loc[[r_index1, r_index2], ["col_1", "col_2"]]` -> access multiple rows and specific colums as dataframe

ROW ACCESS ILOC

- `df_name.iloc[row_num]` -> access the row as dataseries
- `df_name.iloc[[row_num]]` -> access the row as dataframe
- `df_name.iloc[[row_num1, row_num2],[col_num1, col_num2]]` -> access multiple rows and columns as dataframe
- `[start:end]` also possible
- `df_name.iloc[[:],[col_num]]` also possible

COMPARISON and BOOLEAN OPERATORS

- Comparison opeators: `>` , `<` , `==` , `!=` , `>=` , `<=`
- Boolean operators: `and` , `or` , `not`
- numpy array can use the comparison operators to get a boolean numpy array
- for boolean operations on numpy arrays: `np.logical_and()` , `np.logical_or()` , `np.logical_not()`

CONDITIONALS

- syntax:

```
if condition1:
```

```
    expression
elif condition2:
    expression2
else:
    expression3
```

Hidden output

FILTERING PANDAS DATAFRAME

- NOTE: A pandas dataseries can be used as a numpy array : this is how comparision is done

Process:

1. Get a column as a dataseries: `df_name["col_name"]` , `df_name.iloc[:, col_index]` , `df_name.loc[:,"col_name"]`
2. Compare as a numpy array: `ds_filtered > n`, `np.logical_and(ds_filtered > x, ds_filtered < y)`
3. Subset the dataframe: `df_name[compared_nparray]`

LOOP:

while loop

syntax:

```
while condition_is_true:
    looping_expression
    modify_condition
```

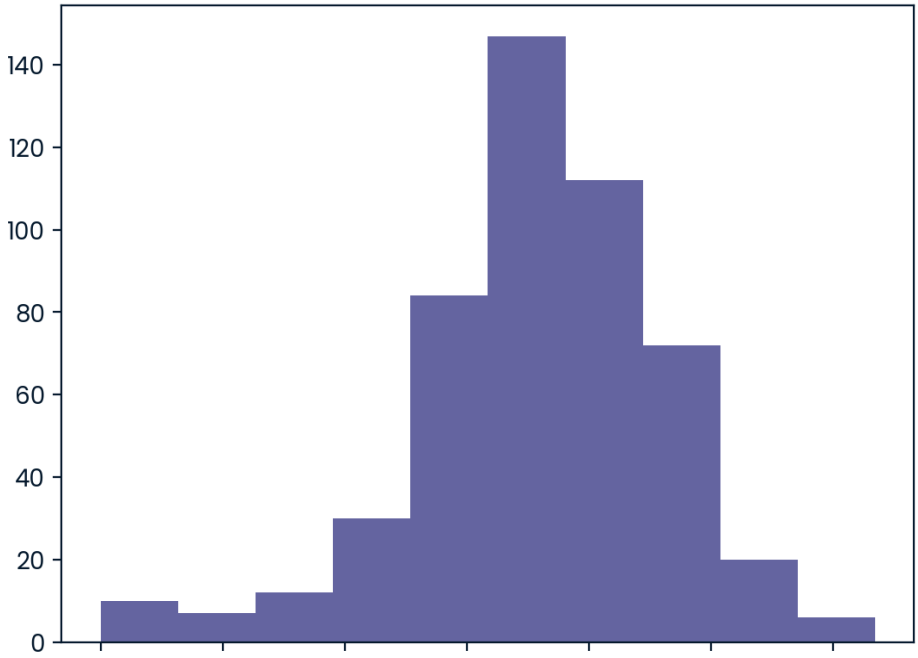
Hidden output

- if infinite loop occurs, then press `control + C` (`^+C`) to terminate the code

for loop

EMPIRE STATE PROBLEM

```
# numpy and matplotlib imported, seed set
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
# Simulate random walk 500 times
all_walks = []
for i in range(500) :
    random_walk = [0]
    for x in range(100) :
        step = random_walk[-1]
        dice = np.random.randint(1,7)
        if dice <= 2:
            step = max(0, step - 1)
        elif dice <= 5:
            step = step + 1
        else:
            step = step + np.random.randint(1,7)
        if np.random.rand() <= 0.001 :
            step = 0
        random_walk.append(step)
    all_walks.append(random_walk)
```

```
# Create and plot np_aw_t
np_aw_t = np.transpose(np.array(all_walks))

# Select last row from np_aw_t: ends
ends = np_aw_t[-1,:]

# Plot histogram of ends, display plot
plt.hist(ends)
plt.show()
```